Resultados de aprendizaje
RAA2 - Aplicar conceptos de gramáticas generativas en la implementación de sistemas de procesamiento de lenguajes especializados para dominios o aplicaciones específicas.
RAA3 - Simplificar gramáticas mediante formas normales para el procesamiento y análisis eficiente de lenguajes.
RAA6 - Comunicar con vocabulario y lenguaje especializado las ideas principales sobre los modelos computacionales estudiados y sus aplicaciones.

Nivel de uso de IAG	Descripción
3. Colaboración con IAG	Los estudiantes pueden apoyarse en la IAG para completar tareas o desarrollar entregables asociados a la actividad, aprovechando las capacidades de estas herramientas para mejorar los productos. Asimismo, se espera que los estudiantes lleven un registro de sus interacciones con la IAG y estén en la capacidad de modificar los resultados generados, demostrando comprensión y dominio conceptual. Este registro debe contener tanto el contenido de autoría propia proporcionado a la IAG como los prompts empleados.

1. Goal

Working in pairs or trios, design and implement a Domain-Specific Language (DSL) to represent the candidate profiles produced by *ResumeLens*. The language must be defined using an EBNF grammar and implemented with textX. The final system must read valid résumé specifications, build their corresponding model, validate their structure, and generate a Markdown visualization.

2. Team and GitHub repository setup

Create a private GitHub repository today, from day 0 of the activity. The repository must keep the development history of this component.

```
resumelens-dsl-<team-name>
```

Add the following collaborators:

- All team members
- Professor: angievig, jennifer.viafara@gmail.com
- Course monitors

Important: each team member must contribute through commits to the repository.

Suggested repository structure

```
resumelens-dsl/
│
├── README.md
├── requirements.txt
│
├── grammar/
│   └── resume.tx
│
├── src/
```

```
    ├── parser.py
    ├── validator.py
    └── markdown_generator.py

├── examples/
    ├── resume_01.resume
    ├── resume_02.resume
    └── resume_03.resume

├── output/
    ├── resume_01.md
    ├── resume_02.md
    └── resume_03.md

├── docs/
    ├── grammar.md
    ├── metamodel.md
    └── diagrams/
            ├── resume_01_tree.png
            ├── resume_02_tree.png
            └── resume_03_tree.png

└── tests/
    ├── valid/
    └── invalid/
```

3. Language design

Design an EBNF grammar capable of representing the structured candidate information produced by ResumeLens.

The language must represent, at minimum:

- Personal information
- Contact information
- Professional experience
- Education
- Skills
- Normalized qualifications
- Result of the candidate profile classification

The grammar must support repeated structures when necessary, for example:

- skills
- professional experiences
- education records
- qualifications

You may define additional structures if they are useful for your language.

4. EBNF specification

Document the formal definition of the language in:

```
docs/grammar.md
```

4.1 Terminals

Identify the lexical elements of the language. For example:

```
STRING
 INT
 ID
 "profile"
 "skills"
 "experience"
```

4.2 Non-terminals

Identify the syntactic structures defined by the grammar. For example:

```
Resume
 PersonalInfo
 Contact
 Experience
 Education
 Skill
 Classification
```

4.3 EBNF rules

Write the complete grammar using EBNF notation. Your documentation must clearly show rules such as:

```
Resume ::= ...
 PersonalInfo ::= ...
 Experience ::= ...
```

5. textX grammar

Implement the language in:

```
grammar/resume.tx
```

The grammar must be executable with textX. For example:

```
from textx import metamodel_from_file

 resume_mm = metamodel_from_file("grammar/resume.tx")
```

The final grammar must correctly:

- accept valid résumé specifications
- reject syntactically invalid specifications
- identify repeated structures

- construct the candidate model

6. Grammar metamodel

Document the metamodel generated by the grammar in:

```
docs/metamodel.md
```

Identify the main concepts/classes generated from the textX grammar and the relationships among them.

```
Resume
├── PersonalInfo
├── Contact
├── Experience *
├── Education *
├── Skill *
└── Classification
```

Include a diagram showing the principal elements of the metamodel and their relationships.

7. Résumé examples

Create at least three different valid résumés written using your DSL and store them in:

```
examples/
resume_01.resume
 resume_02.resume
 resume_03.resume
```

The examples must not be simple copies of the same structure. They should exercise different characteristics of the language, such as multiple experiences, skills, education records, or different profile classifications.

8. Parse trees / model structure diagrams

For each résumé example, generate a representation of its parsed structure. Store the diagrams in:

```
docs/diagrams/
```

Minimum required:

```
resume_01_tree
 resume_02_tree
 resume_03_tree
```

The diagrams must make visible how the résumé is decomposed according to the grammar. You should distinguish between the grammar, the metamodel, and the parsed model.

```
Resume
├── PersonalInfo
│   ├── name
│   └── location
├── Contact
│   ├── email
│   └── github
```

```
├── Experience
│   ├── position
│   ├── company
│   └── years
├── Skills
│   ├── Python
│   ├── Pandas
│   └── SQL
└── Classification
    └── MachineLearningEngineer
```

9. Validation

Create test cases demonstrating that the grammar can distinguish between valid and invalid candidate profiles.

```
tests/
├── valid/
└── invalid/
```

Include invalid examples with errors such as:

- missing mandatory information
- incorrect keywords
- malformed structures
- invalid nesting
- incorrect list syntax
- incomplete candidate profiles

Your program must demonstrate that textX rejects the invalid cases.

10. Markdown visualization

Once a résumé has been successfully parsed, generate a readable Markdown representation using the resulting model.

Implement this functionality in:

```
src/markdown_generator.py
```

Example output:

```
# Wednesday Addams

## Candidate Profile

**Classification:** Software Engineer

## Contact

- Email: wednesday@nevermore.edu
- Location: Jericho, Vermont
```

```
## Skills

- Python
- Git
- PostgreSQL
- Machine Learning

## Experience

### Investigative Software Developer

**Nevermore Academy**

Developed data analysis tools to investigate unusual events around Nevermore.

## Education

**Nevermore Academy**

Specialization in supernatural investigations.
```

Generate one Markdown file for each example:

```
output/
├── resume_01.md
├── resume_02.md
└── resume_03.md
```

11. README requirements

The root README.md must contain at least the following sections:

- Project description: role of the DSL inside ResumeLens
- Team: names and GitHub users
- Language: brief description of the candidate profile language
- Project structure: explanation of the main folders
- Requirements: Python version and required libraries
- Installation instructions
- Instructions to run the parser
- Instructions to generate Markdown
- Description of the three candidate profile examples
- Authors

Minimum execution examples:

```
pip install -r requirements.txt

python src/parser.py examples/resume_01.resume

python src/markdown_generator.py examples/resume_01.resume
```

12. Expected work during today's class

By the end of today's class, each team should have:

1. Created the private GitHub repository and added all collaborators.
2. Defined the main concepts of the candidate language.
3. Identified terminals and non-terminals.
4. Produced a first version of the EBNF grammar.
5. Created the initial resume.tx textX grammar.
6. Written at least one valid résumé example that can be parsed by textX.

13. Homework / final completion

Complete and refine the remaining elements:

- final version of the grammar
- metamodel documentation and diagram
- three résumé examples
- three parse-tree/model-structure diagrams
- valid and invalid tests
- Markdown generator
- README and final documentation

14. Minimum final evidence

The repository must demonstrate the complete pipeline:

```
Candidate information
    ↓
Résumé DSL file
    ↓
textX grammar
    ↓
Metamodel / parsed model
    ↓
Validation
    ↓
Markdown visualization
```

Completion criterion: a project that only defines the grammar but does not successfully parse candidate profiles and generate the Markdown visualization is incomplete.